

TRENTOS-M Demo Network Filter Concept

Page Content

- [Overview](#)
- [Possible Network Filter ideas](#)
 - [Variant A - Application Level Firewall \(a.k.a Proxy Firewall/Application Gateway\)](#)
 - [Possible ideas for filters:](#)
 - [TLS Gateway for unprotected interfaces](#)
 - [Web Application Firewall](#)
 - [Data Loss Prevention](#)
 - [Filter for GPS coordinates matching a specified binary protocol](#)
 - [How should TLS be integrated into this variant?](#)
 - [TLS-Termination Proxy](#)
 - [TLS-Forward Proxy](#)
 - [Variant B - Packet Filtering](#)
 - [Basic stateless packet filtering](#)
 - [More elaborate packet filtering](#)
 - [Possible actions that could be implemented for a packet filter, regardless if stateful or stateless](#)
 - [Open Issues](#)
 - [Additional design choices to consider](#)
 - [Variant C - Merge both filter variants into one filter demo](#)
- [Selected Filter Demo](#)
 - [GPS coordinates filter](#)
 - [Architecture](#)
 - [Hardware Setup](#)
 - [QEMU Setup](#)
 - [Run in the Host Network](#)
 - [Run in the Docker Bridge Network](#)
 - [Selected Docker Setup](#)
- [Possible demo extensions](#)
 - [Add an Admin Webserver to configure the demo system](#)
- [Development Work Packages](#)
 - [Epics](#)
 - [Epic: Create TRENTOS Network Filter Demo](#)
 - [Future Tickets](#)

[DPB-46](#) - Getting issue details...

STATUS

[SEOS-2218](#) - Getting issue details...

STATUS

Overview

This page will eventually contain the complete documentation for the new Network Filter Demo, which is planned to be added to TRENTOS-M with v1.3.

The main feature of the demo shall be filtering of Network packets based on a property (to be discussed):

- only certain protocols may pass
- only certain ports may pass
- only certain MAC addresses or IP addresses may pass
- Filter content (e.g. replace text in payload or block packets based on content)
-

This demo would be best suited on boards with 2 Ethernet ports

- Nitrogen6SoloX
- Run in QEMU with the two NIC Drivers
- RaSpi or Sabrelite with Ethernet over SPI extension

Possible Network Filter ideas

Variant A - Application Level Firewall (a.k.a Proxy Firewall/Application Gateway)

- Operate at the application level.
- Filter ingress traffic for possible attacks.
- Filter egress traffic for sensitive data.
- Filter traffic only with regards to the application (or service) for which they are intended.

- Full reverse proxy architecture that serves as a termination point for the ingress traffic.

Possible ideas for filters:

TLS Gateway for unprotected interfaces

This could be a first draft of the "Maintenance interface protection", as discussed with Hensoldt.

- ETH-INT: We have one interface without TLS that goes to an unprotected device
- ETH-EXT: We have one interface with a TLS Connection
- The filter only accepts TLS connections from ETH-EXT
 - If valid, a connection is established on ETH-INT as non-TLS
- The filter configurable allows/disallows Connections initiated from ETH-INT to ETH-EXT
- Optional the ETH-EXT to ETH-INT could allow only certain protocols.
- This setup can be seen in the diagram below showcasing a "TLS-Termination Proxy" architecture.

Web Application Firewall

- Filter on allowed HTTP verbs (e.g. only allow "GET" Requests).
- Possible library that was already previously used in a project that wasn't further pursued <https://github.com/nodejs/http-parser>
- Filter against Injection attacks?
 - SQL-Injections based on a known attacks to filter against:
 - <https://github.com/danielmiessler/SecLists/blob/master/Fuzzing/SQLi/quick-SQLi.txt>

Data Loss Prevention

- Demonstrate how sensitive data can be protected for e.g. with de-identification (similar to what Pontus has implemented with his Message Gateway for MQTT messages) https://bitbucket.hensoldt-cyber.systems/projects/RA/repos/message_gateway/browse.
- Filter would be masking sensitive data by partially or fully replacing characters with a symbol, such as an asterisk (*) or hash (#).
- Could be applied either on ingress or egress traffic.

Additional services that could be interesting to filter on

- SMTP
 - E.g. discard any mail attachments
- FTP
 - E.g. enforce the usage of passive mode

Filter for GPS coordinates matching a specified binary protocol

Idea would be that there are (at least) three isolated domains:

1. Egress TCP Connection domain (Black domain)
2. Ingress TCP Connection domain (Red domain)
3. Filter component; connecting those two (Red domain)

The domains need to be isolated.

The data flow shall only be possible from Red to Black domain.

The data transferable from red to black needs to be structured information (no text, only encoded values).

The structure of the information shall be specified with a simple binary protocol.

Based upon the protocol, only valid information shall pass to the black domain.

If unspecified data is received in the Ingress connection, this data shall be logged.

For this setup, two applications on a PC need to be implemented:

- Sending application: Sends...
 - ...data specified by protocol
 - ...data not specified
 - receives log messages from Filter demo
- Receiving data: Receives data in protocol

Configuration can be performed statically in the filter demo (no Proxy features necessary yet).

Proposal for protocol (GPS coordinates):

4bytes BE MessageType (encoded; 23=WGS84 based coordinates);

Float longitude [-180;180], degree

Float latitude [-90;90], degree

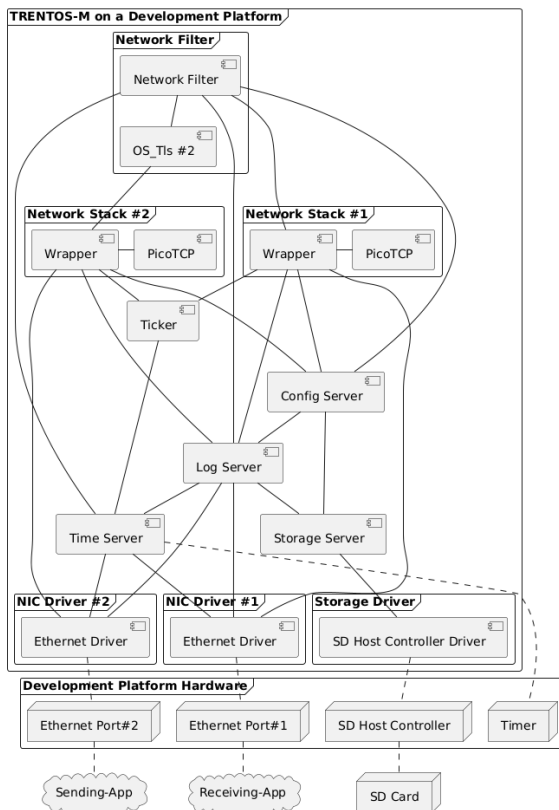
Float altitude [-1000;10000], m

How should TLS be integrated into this variant?

TLS-Termination Proxy

Open Issues:

- Our TLS API does not support setting up a TLS server that listens on incoming connections. If we would include TLS in this demo, would need to filter the egress data traffic.

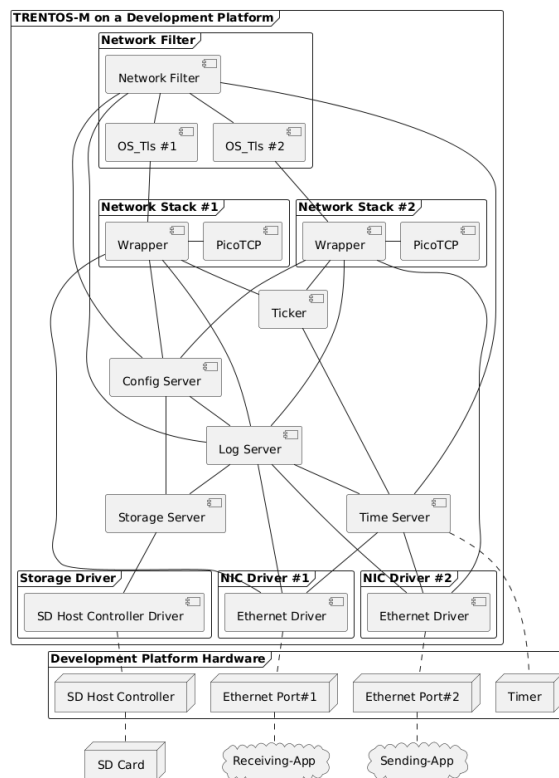


Network Filter Demo TLS-Termination Proxy - CAMkES Architecture

TLS-Forward Proxy

Open Issues:

- How do we handle authentication in the internal TLS connection?
- Our TLS API does not support setting up a TLS server that listens on incoming connections.



Network Filter Demo TLS-Forward Proxy - CAMkES Architecture

Variant B - Packet Filtering

- Layer 2
 - MAC Address Filter, As a byproduct of this filter, it could also be used as a baseline for the virtual network switch required for [Virtualization Prototype](#). [SEOS-780](#) - Getting issue details... [STATUS](#)
 - VLAN Tag filter, see https://de.wikipedia.org/wiki/IEEE_802.1Q
- Layer 3/4 packet-filtering making processing decisions based on network addresses, ports, or protocols.
 - IP-Address filter
 - TCP/UDP port filter

Basic stateless packet filtering

Stateless filtering assumes that information in the packets can be trusted. An attacker could send SYN/ACK to the network as an initial packet and it would pass the firewall.

- Source IP Address
- Destination IP Address
- Transport protocol
- Source and destination application port
- Specific protocol flags (e.g. TCP's ACK- and SYN-flag)
- Filter based on the network interface a packet has been received on.

Protocols like UDP do not have useful state information, so the protocol to focus on would be TCP.

Advantages in regards to implementation:

- Simpler to implement than stateful filtering
- Higher performance (but probably not a very relevant attribute in the context of the demo)
- Compared to [Variant A](#), packet filtering does not need to bother with any topics related to TLS so nothing of this would require server side implementation of TLS that we do not yet feature.

More elaborate packet filtering

- Stateful or dynamic packet filtering
 - For example incoming UDP packets would only be allowed to pass through if they are responses to outgoing UDP packets that have been observed
 - Another example would be to only accept TCP packets with the SYN bit set only as part of TCP connection initiation
- Protocol checking
 - For example not allowing HTTP transfer to specific sites

As mentioned in the advantages of the simple packet filtering, stateful filtering will be more complex to implement. Why could it still be interesting to consider?

- Allows to check the protocol state (like SYN, SYNACK etc.) in the firewall
- Can detect replies even in case of UDP usage. Therefore, we can define also for UDP who is allowed to initiate a communication.
- By tracking connections and connection requests of a host, it is also possible to add rate limiting etc.
- Important feature is connection tracking.
 - Firewall sees first packet of connection.
 - Subsequent packets are considered part of this connection.
- States that should be implemented for connection tracking:
 - New
 - First packet for a yet unknown connection
 - Established
 - Packet for an existing connection
 - Invalid
 - Packets contain invalid header fields
 - Related
 - Not necessary for a more basic implementation as this also will require application-specific configuration of the firewall so it can understand which packets are related in the context of a specific application. Certainly an overkill for a demo.

Possible actions that could be implemented for a packet filter, regardless if stateful or stateless

- Pass the packet
- Drop the packet
- Log the passed or dropped packet (entirely or only parts deemed relevant of it)

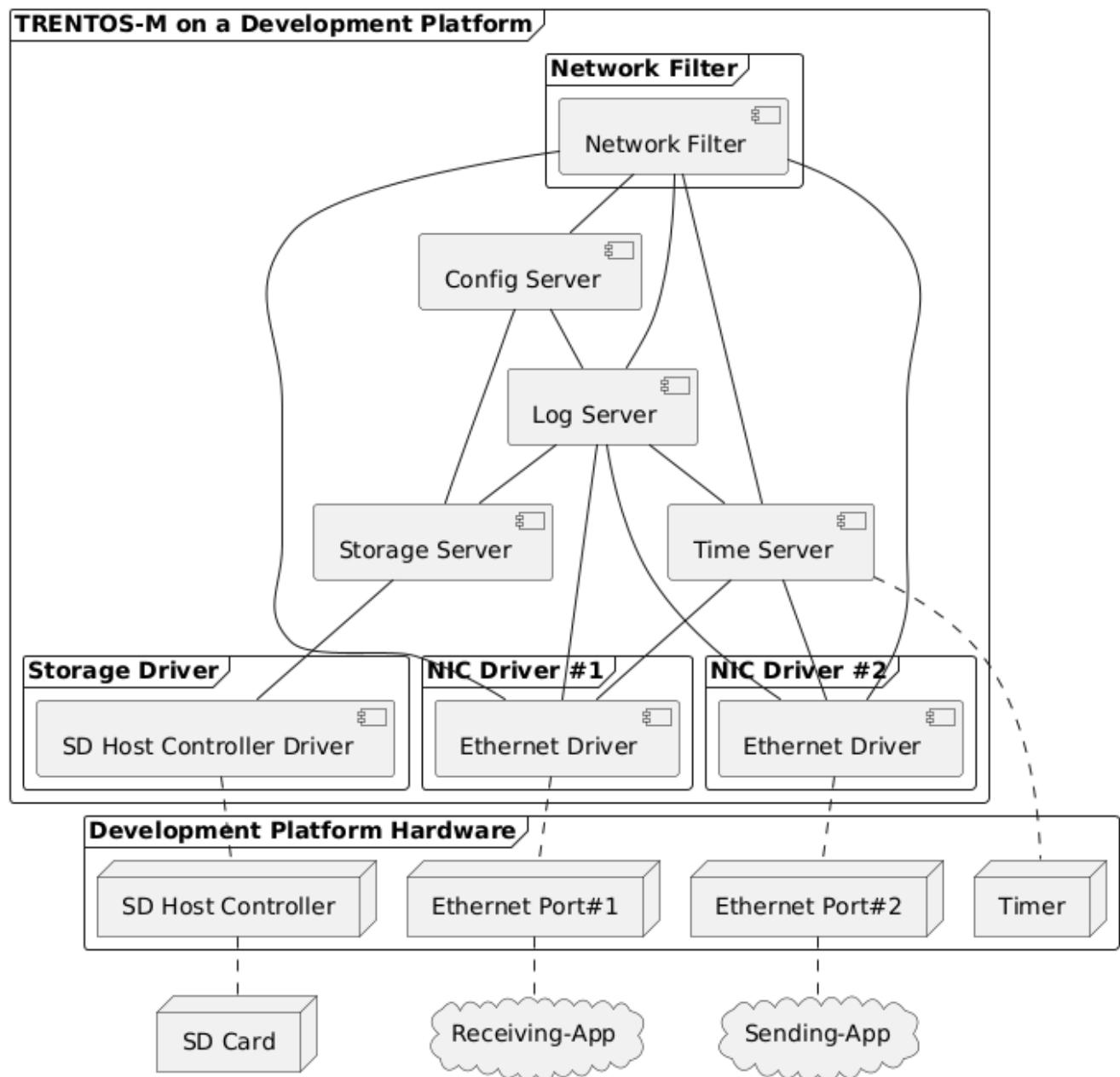
Open Issues

- PicoTCP does not support creating RAW-Sockets (compared to some other NwStacks we are looking into that already provide this feature) or something similar to PACKET-Sockets, which means that the filter component would be placed directly on top of the NIC driver. A GPL fork of picoTCP seems to have added support to the stack for RAW and PACKET sockets (<https://github.com/virtualsquare/picotcp/commit/9142e028457e333914df214d3930f464eb548a5e#diff-69a91de09281e1d715da54cc44989b10b3818b567e2f741a656154a27b2f90e9>) so if we would like to extend our version with these types of sockets, we could check out what was done there.
- As we probably don't want to create this whole frame analyzing codebase on our own, this might require the usage of some 3rd-Party library to process the frames and allow our filter logic to be relatively compact.
 - Check the following libraries/tools if they have helpful hints for this approach:

<https://www.tcpdump.org/> (pcap)

<https://github.com/libnet/libnet> (libnet)

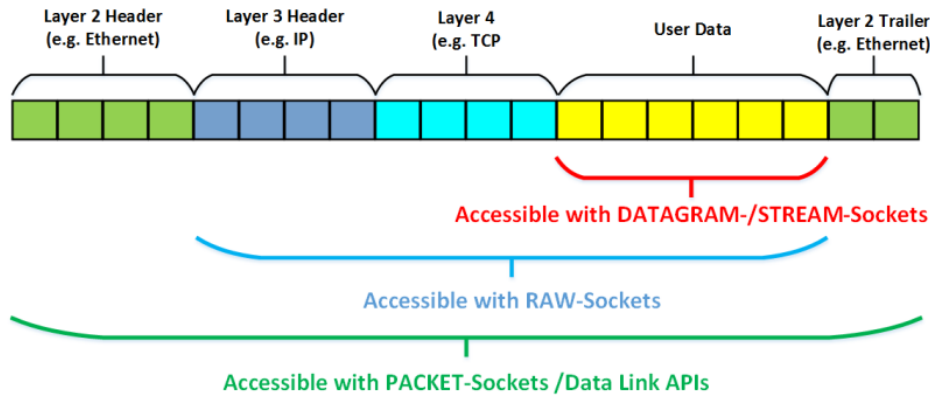
- Depending on the system architecture and the requirements, the filter component might need to be able to issue ARP replies.



Network Filter Demo Packet Filter - CAMkES Architecture

Additional design choices to consider

Implement component that connects to the NIC_Driver component and abstract a raw socket interface to the higher layer.

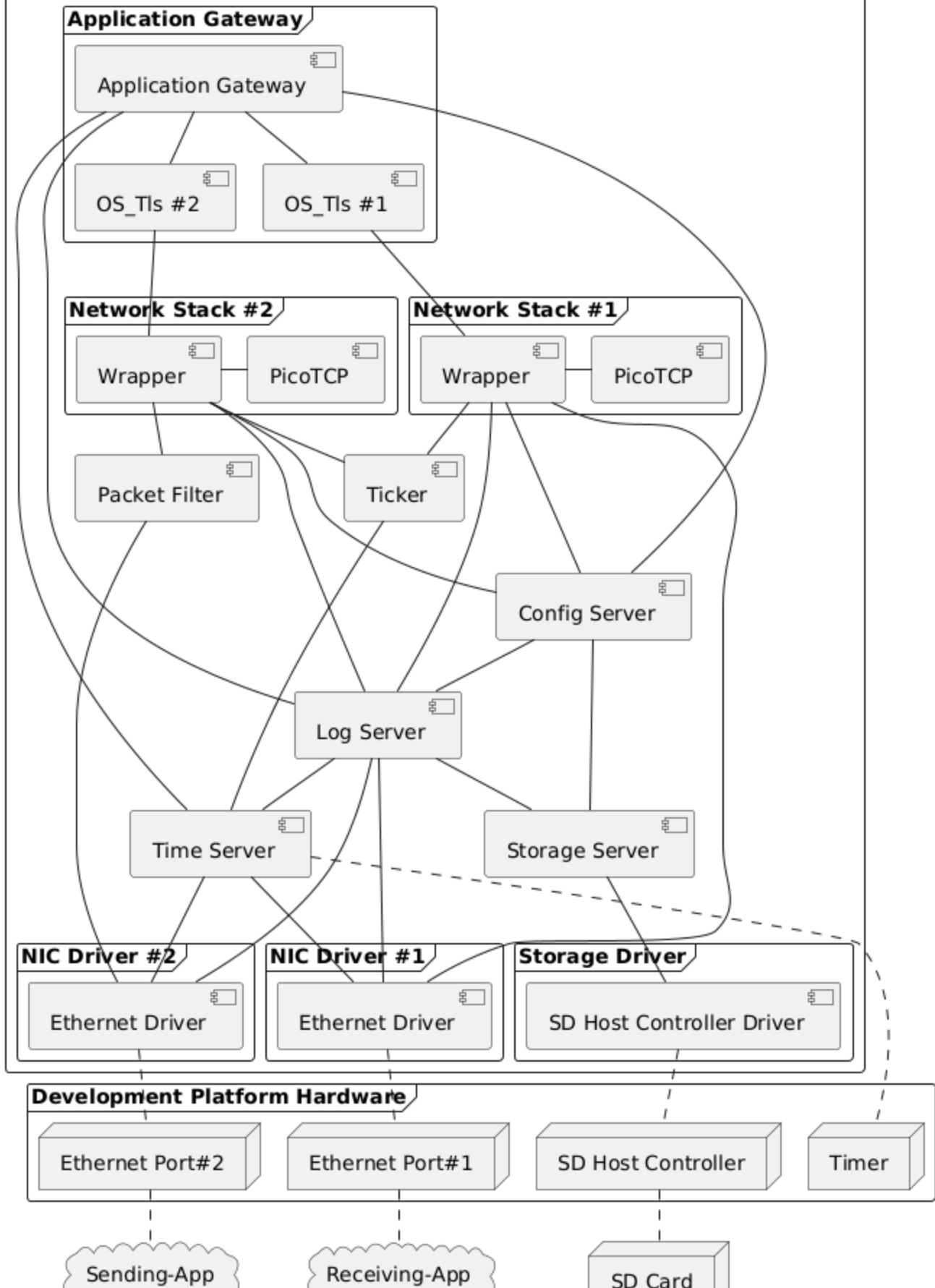


Our current NIC interface is very basically quite close to what Linux would describe as a PACKET-Socket. Creating a very small component that sits on top of the NIC driver and offers a socket interface to the upper layer that offers both RAW sockets and PACKET sockets, would offer a very portable and generic solution to process network traffic. If we realize that we have several use-cases for RAW or PACKET-Sockets, we could consider creating this small component that would then be reusable for other systems.

Variant C - Merge both filter variants into one filter demo

Another extension that could be done for the demo is to essentially merge the Application Gateway (Variant A) and the Packet Filter (Variant B) into one system that filter on both layers. The initial packet filtering would then be done directly on top of the NIC_Driver receiving the incoming Sending-App packets. All packets that do not get filtered out, are presented to the NetworkStack above and can be further filtered by the application layer filter component:

TRENTOS-M on a Development Platform





Merge Network Filter Demo Packet Filter - CAMkES Architecture

Why extended the demo by merging both filter variants?

- All in one system that would showcase not only packet filtering but also application layer filter. This would could increase the chance that customers can find something interesting in this demo system to pick out for their own projects.

Why not directly implement this option instead of the variants presented above?

- It adds additional complexity to the system and we might end up with a similar situation we have with the IoT demo. The demo might be interesting but simply too overloaded with features.
- If we don't have enough time, we might end up with two filters that both are rather poorly implemented instead of one filter variant that already has a higher level of maturity.

Selected Filter Demo

The following sections summarize the scope of the agreed upon filter demo system. This scope was derived from the ideas and points gathered in the previous sections.

GPS coordinates filter

Three isolated domains:

1. Egress TCP Connection domain (Black domain)
2. Ingress TCP Connection domain (Red domain)
3. Filter component; connecting those two (Red domain)

The data flow shall only be possible from Red to Black domain.

The data transferable from red to black needs to be structured information (no text, only encoded values).

The structure of the information shall be specified with a simple, binary protocol.

Based upon the protocol, only valid information shall pass to the black domain.

If unspecified data is received in the Ingress connection, this data shall be logged.

For this setup, two applications on a PC need to be implemented:

- Sending application: Sends...
 - ...data specified by protocol
 - ...data not specified
 - receives log messages from Filter demo
- Receiving data: Receives data in protocol

Configuration can be performed statically in the filter demo (no Proxy features necessary yet).

Proposal for protocol (GPS coordinates):

4bytes BE MessageType (encoded; 23=WGS84 based coordinates);

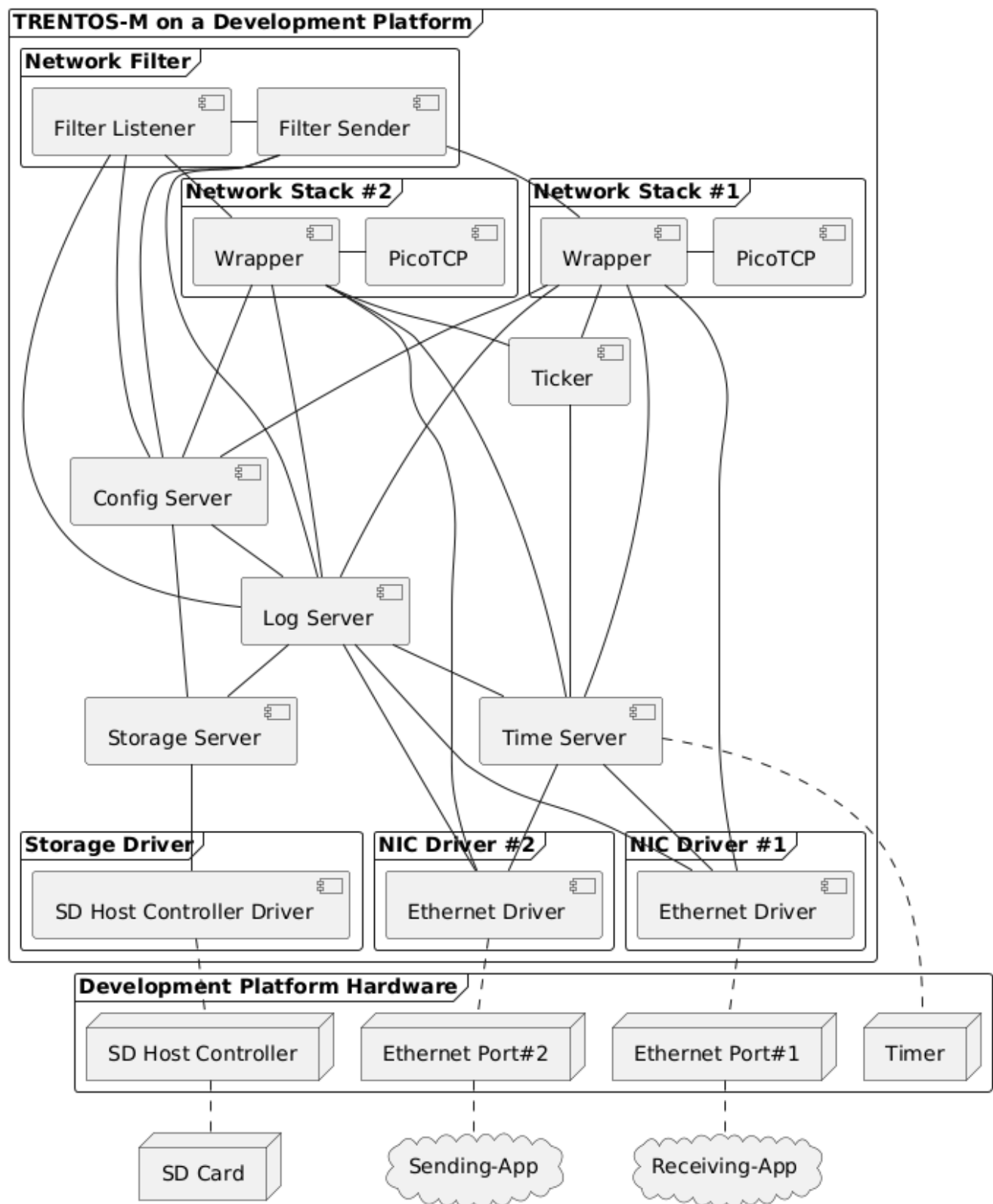
Float longitude [-180;180], degree

Float latitude [-90;90], degree

Float altitude [-1000;10000], m

4B Checksum (MD5)

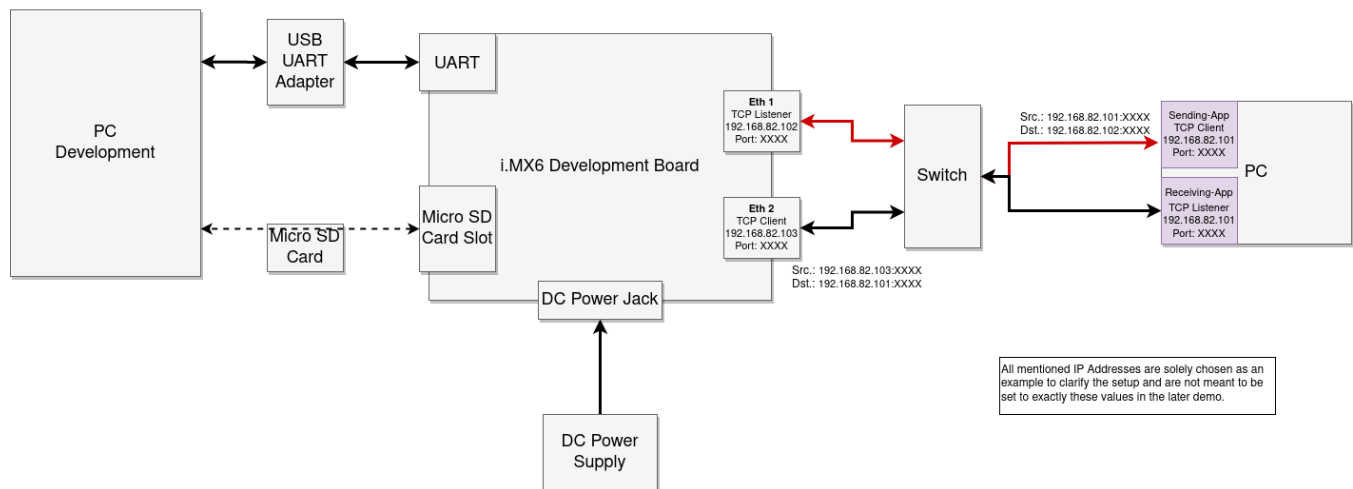
Architecture



Network Filter Demo Application Layer Firewall - CAMkES Architecture

Hardware Setup

- Guiding ideal is that the new demo does not require additional HW for the user. This will keep our "minimal" required setup to run our demos compact.

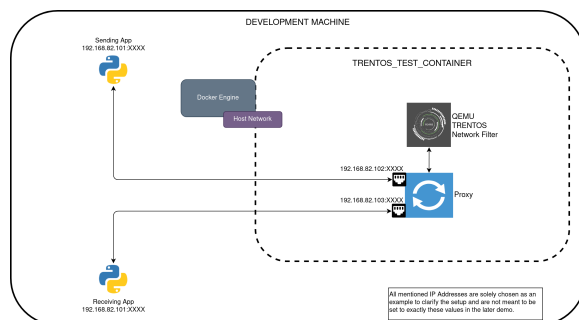


QEMU Setup

The following example implementation options both have the Python Applications running outside of the container. This preliminary design choice was done due to the following points:

- Running the demo will require to attach to a running container, which will make the run steps more complicated and getting to the point to run the Python Apps and monitor all generated output (Sending-App, Receiving-App, TRENTOS-Filter)
- The transition from the QEMU to the real HW Setup is much closer if the Python Apps run natively
- The possible advantage of pre-installing the required Python environment in the Docker container is irrelevant, since we want the demo to be compatible with the v1.2 Docker images.
- Debugging with Wireshark will be complicated

Run in the Host Network



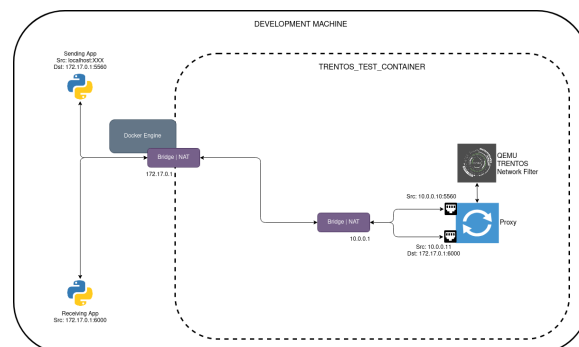
Advantages

- Setup is very close to the HW setup, so the setup steps between the demo on real hardware and in QEMU should be more aligned.
- Not NAT settings required for the docker bridge.

Disadvantages

- Demo not running in an isolated container namespace which might cause issues when running it in CI.
- Needs to be configured by the user of the Demo so the Network Stacks have valid IP addresses in the context of the users development network.
- Less control over how exactly it runs on the users development machine and therefore more error prone in setup.
- The setup and the removal of the TAPs should be moved into separate scripts (currently automatically set up and removed by the usage of the docker container with the entrypoint script)

Run in the Docker Bridge Network



Advantages

- Better overall control over the environment setup in which the demo will run for the customer as there should be no adaptations needed on user side.
- Demo runs in a isolated environment which will make it a bit easier to run it in CI.
- Creation and removal of required TAP devices is automatically taken care of in the container environment
- Can be delivered completely preconfigured without any need to set specific IP addresses for the Network Stack components that match with the Host Network.

Disadvantages

- NAT settings required to forward the traffic at the docker bridge to the bridge created by us that the TRENTOS application is connected to.
- Since we want to be backward compatible and also have it run for the v1.2, changing the NAT settings would need to be done

shipped with the demo and either need to be run manually or we will need some more automation to take care of this setup step.

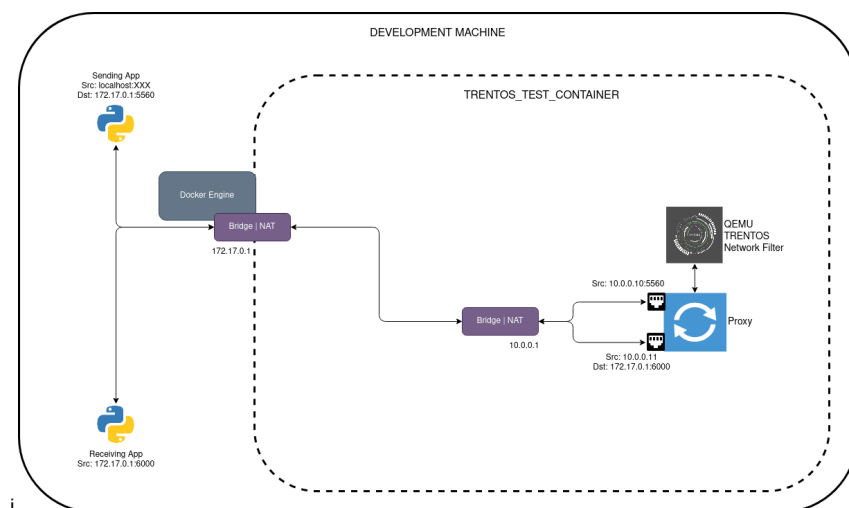
- We would probably also have to create some new endpoint script that is used to overwrite the existing one in the test container as a quick test of switching from the docker bridge to the host network when starting the container produces errors (starting mosquito and nginx both result in failures due to name resolution issues).

by overwriting the current endpoint script that every container executes on start. This can be done by utilizing the cmd line option

--entrypoint [new_command] and pass a new script to execute that will set up the required environment for the demo (TAP device creation and setting of NAT rules).

Selected Docker Setup

Considering all the presented advantages and disadvantages regarding the possible Docker environment setup for the Demo, the option to run the container on the docker bridge network was selected. The demo should therefore run in the setup detailed in the diagram below:



- The chosen ports shown above will be the default config but the demo should provide documentation on how to adapt these setting in case the user is running some process locally that is already using these default ports.
- The IP Address of the container that the Sending-App should connect to will depend on how many containers the user is running (the first container running on the host will start with 172.17.0.2 and increment with every added container), so this will need to be configured before the demo can run.
- The new endpoint script that needs to be created for the demo will need to setup the required NAT rules to forward the packets to the bridge network the Proxy Interfaces are connected to.

Possible demo extensions

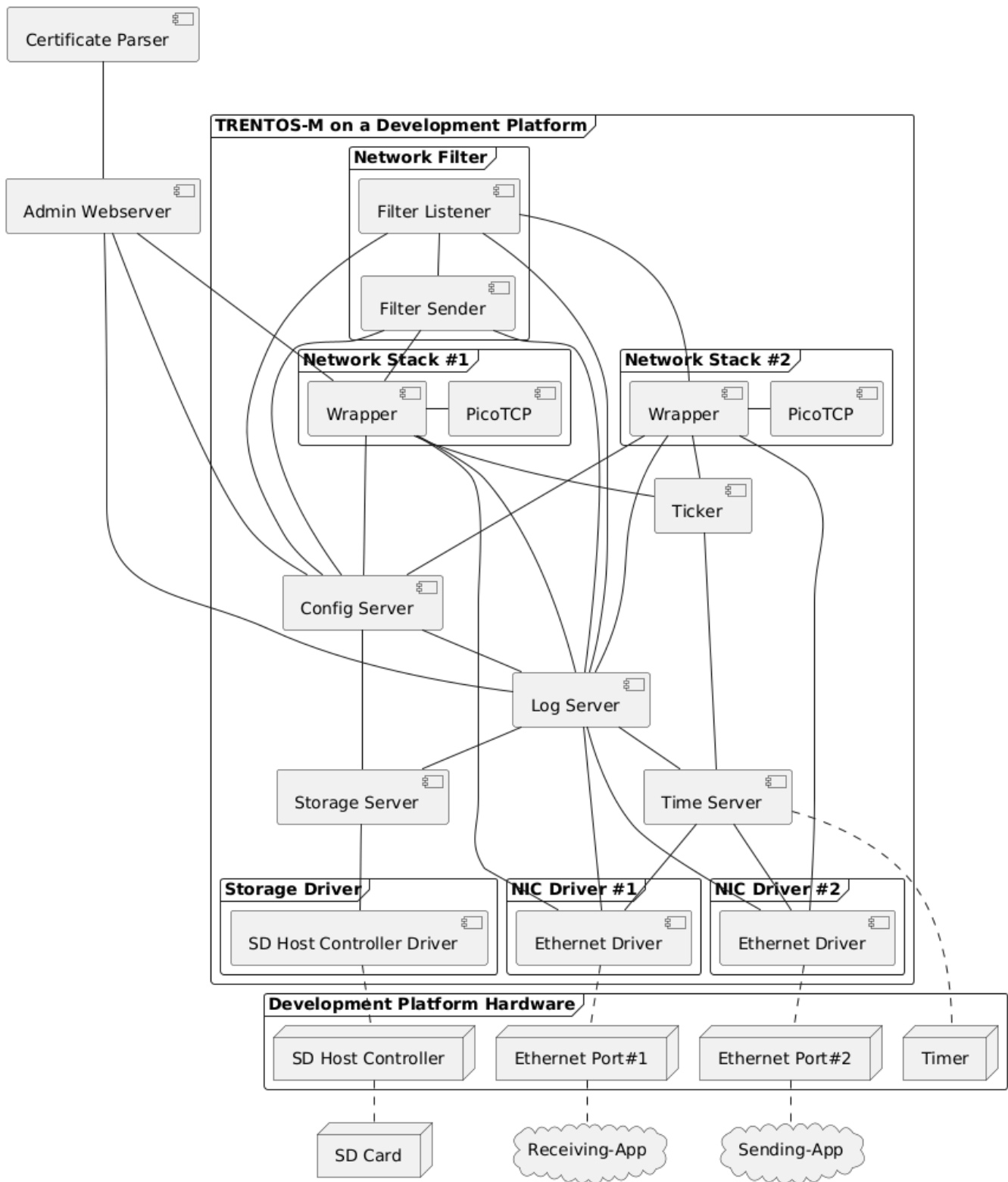
Add an Admin Webserver to configure the demo system

A clear input from sales side regarding this whole demo is that ideally it should have some way that will allow the user to possibly log on to the running system and change the filtering configuration or maybe read out the logs. Essentially showcasing how some form of interface could also be realized with this particular use-case.

One possible idea that would come to mind here is to have some Webserver component running in the system that would listen on a specific port and offer up a webpage (similar to every typical network router device or similar) to a connected client. This page would then need to include some functionality to authenticate and read out log files or change existing filter configuration.

Authentication would require the connected client to present an accepted certificate that will be parsed and verified by a CertParser component connected to the component running the Webserver.

Extending the existing filter demo with such a setup could result in the following example architecture (example showcases the extension of the application-layer filter variant):



Extended Network Filter Demo Packet Filter - CAMkES Architecture

Noteworthy libraries to further investigate for the creation of an embedded webserver:

Library	Link	License
Mongoose - Embedded Web Server / Embedded Networking Library	https://github.com/cesanta/mongoose	GNU GPL v.2

lighttpd - secure, fast, compliant and very flexible web-server which has been optimized for high-performance environments.	https://github.com/lighttpd/lighttpd1.4	BSD-3-Clause
Monkey - fast and lightweight Web Server for Linux. It has been designed to be very scalable with low memory and CPU consumption, the perfect solution for Embedded Linux and high end production environments.	https://github.com/monkey/monkey/	Apache License 2.0
libmicrohttpd - small C library that is supposed to make it easy to run an HTTP server as part of another application.	http://www.gnu.org/software/libmicrohttpd/	GNU LGPL v2.1

Why extended the demo with this admin webserver?

- Offering some option here to reconfigure the "firewall" that this system essentially would be will make for a more compelling demo as this will come closer and closer to a complete product that some customers might already be considering to develop could open doors for project that would require us to implement such a product for a customer.
- It showcases the new CertParser component in a demo.

Why not directly implement the demo with this Admin interface?

- Adding this additional webserver component will of course require a lot more work and testing to reach a required level of maturity.
- It is not strictly needed to be able to showcase how a network filter could be implemented, it would be more of a nice to have but the filter demo itself would already cover enough ground to resemble a real use-case.

Development Work Packages

Existing related backlog tickets:

- [SEOS-780](#) - Getting issue details... STATUS
- [SEOS-1396](#) - Getting issue details... STATUS

Epics

Type	T	Key	Status	P	Summary	Resolution	Assignee
Authenticate to retrieve your issues							
No issues found							

Epic: Create TRENTOS Network Filter Demo

Type	T	Key	Status	P	Summary	Resolution	Assignee
Authenticate to retrieve your issues							
No issues found							

Future Tickets

Ticket Type	Ticket Name	Ticket Description	Belongs to Epic	Current State
Story	Use CRC32 instead MD5 in demo_network_filter	The demo currently used the MD5 algorithm for the checksum calculation. Considering that the payload is only 16 bytes and the hash itself is also 16 bytes, this introduces a large overhead to the messages. The scope of this tasks should be to make the demo use CRC32 instead of MD5. The SDK currently does not support this algorithm which means to complete this task, this will also include agreeing on if and where this algorithm should be added to the SDK or instead just be added in a utils section in the demo itself. We already have a CRC32 implementation in: https://bitbucket.hensoldt-cyber.systems/projects/HC/repos/bmrbl/browse/src/util.c DoD - Clarified how the CRC32 functionality should be integrated into the demo (either directly in demo or in SDK) - Python scripts and TRENTOS system switched to CRC32 - In case documentation already exists, this is updated accordingly.		
Story	Expand filtering features of the filter component	Add more filter logic and possible rules to files traffic once the basic filter works with the whole demo setup. DoD - Additional filter functionality implemented - Additional filters tested	SEOS-2165 - Getting issue details... STATUS	
Story	Extend the Filter Demo with platform support to run it on the BD-SL-i.MX6	NOT CONSIDERED FOR V1.3 Extend the Filter Demo with platform support for the BD-SL-i.MX6. TBD: How will we connect the additionally required Ethernet Port? TBD: (Depending on at which point we take up this ticket) As the Filter Demo in this stage will not yet write the logs to the SD card and won't make use of the ConfigServer to retrieve the configuration, no StorageServer and SdHostController will be required for the scope of this ticket. Adding support for this HW platform should ideally not require a new repository/separate project. We should aim to keep the demos in one repository that builds for all supported platforms as long as possible. DoD - Demo can be built for the BD-SL-i.MX6 - Demo runs successfully	SEOS-2165 - Getting issue details... STATUS	
Story	Extend the Filter Demo with platform support to run it on the RPi3B+	NOT CONSIDERED FOR V1.3 Extend the Filter Demo with platform support for the RPi3B+. TBD: How will we connect the additionally required Ethernet Port? TBD: (Depending on at which point we take up this ticket) As the Filter Demo in this stage will not yet write the logs to the SD card and won't make use of the ConfigServer to retrieve the configuration, no StorageServer and SdHostController will be required for the scope of this ticket. Adding support for this HW platform should ideally not require a new repository/separate project. We should aim to keep the demos in one repository that builds for all supported platforms as long as possible. DoD - Demo can be built for the RPi3B+ - Demo runs successfully	SEOS-2165 - Getting issue details... STATUS	